Lab 3 Optimizations

Muthu Adithya Ramnarayanan

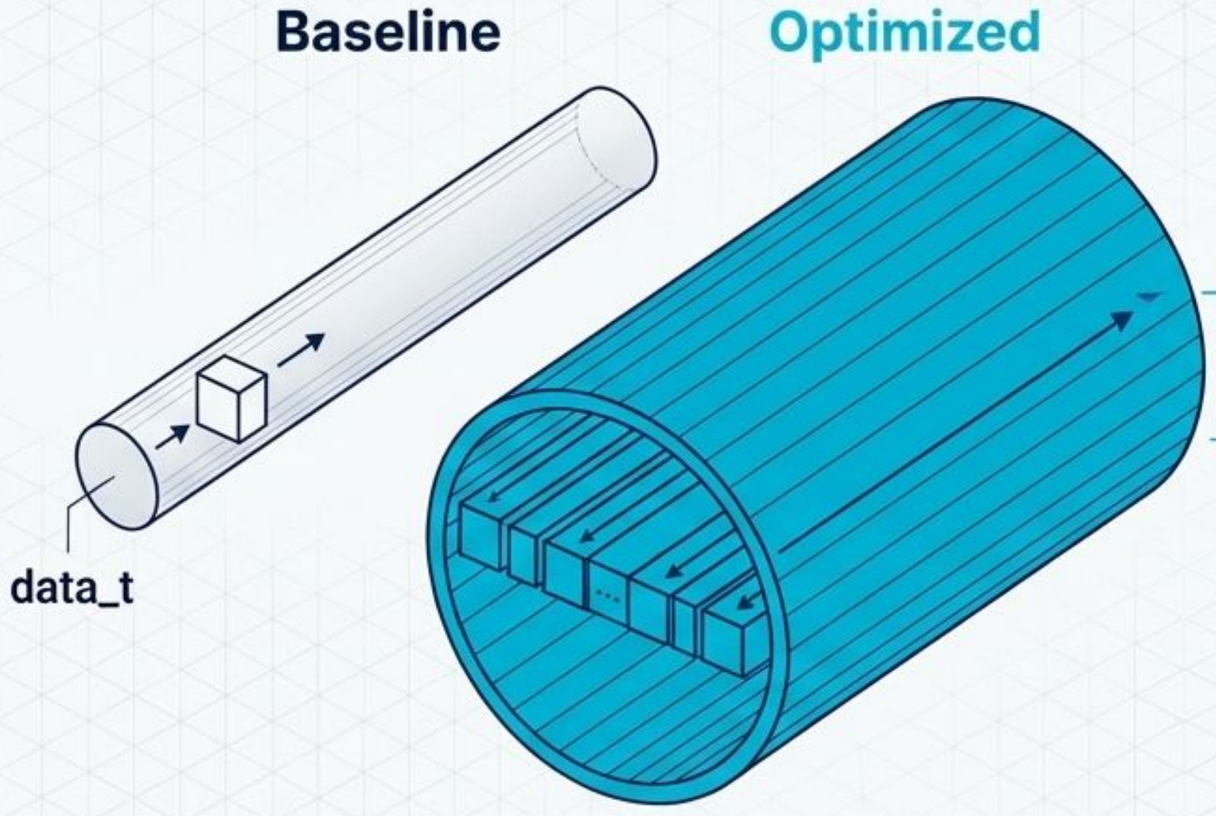
The Baseline Bottleneck: Sequential Array Passes

Functional in software, but devastating for hardware timing and throughput. We must eliminate intermediate RAM storage.

The Paradigm Shift: Software vs. Silicon Mindset

Dimension	Baseline (Software Mindset)	Optimized (Hardware Mindset)
Execution Model	Sequential loops	**Streaming Dataflow (hls::stream)**
Memory Transfer	Large static arrays (BRAM)	**Shallow localized FIFOs (SRL)**
Scaling & I/O	1 element per clock	**32 elements per clock (SIMD)**
Data Types	32-bit standard types	**Custom narrow bit-widths**
Math Operations	Multipliers & Dividers	**Bit-shifts**, **LUTs**, and **Adder Trees**

Overcoming the Memory Wall via Ultra-Wide AXI Vectorization



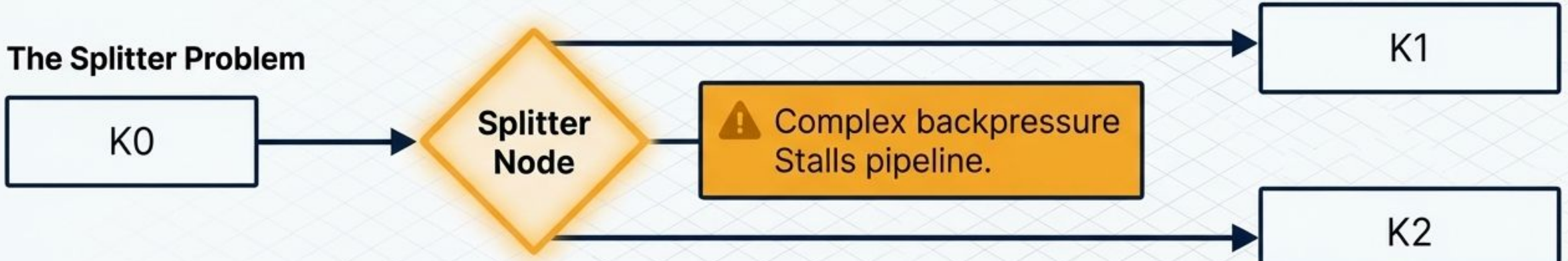

```
typedef ap_uint<1024> ultra_wide_t;
struct vec32_t { narrow_t data[32]; };

#pragma HLS ARRAY_PARTITION variable=vec.data complete
```

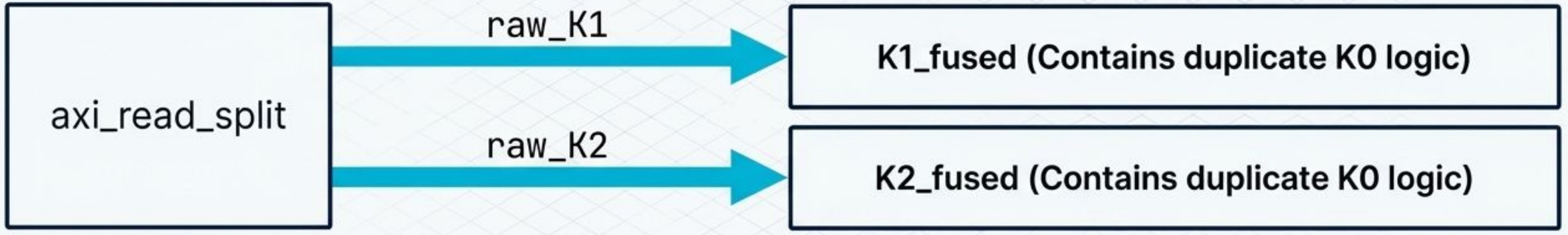
1. **Pack**
   Group 32 elements into vec32_t.
2. **Partition**
   Map to 32 discrete registers instead of a single RAM block.
3. **Unroll**
   #pragma HLS UNROLL by VEC_SIZE (32) divides total execution cycle count by 32.

Resolving Stream Backpressure Through Kernel Fusion

The Splitter Problem

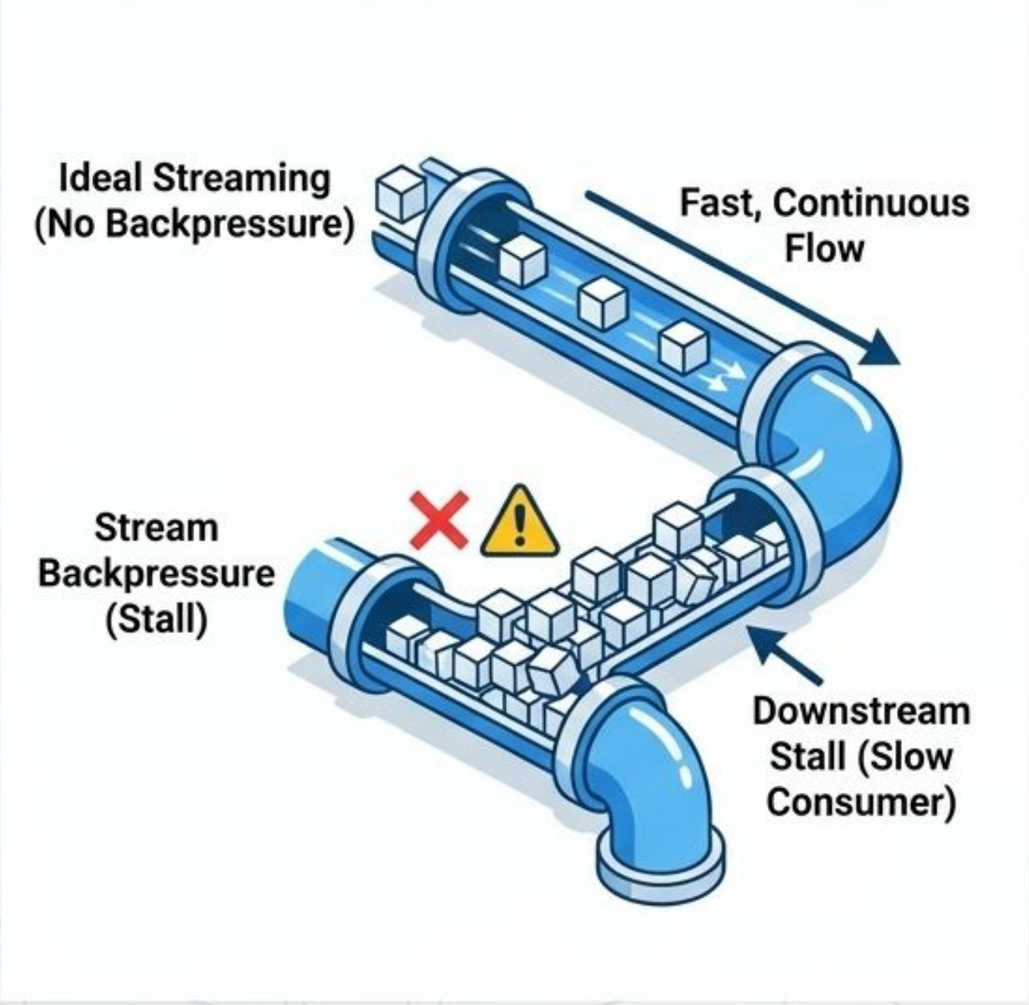


The Fused Solution



Why it works: K0's logic (`x - (x >> 3) + 0.125`) is so lightweight that duplicating its combinatorial logic is practically free in hardware. This entirely eliminates the physical K0 kernel and associated stream synchronization bottlenecks.

Understanding Stream Backpressure

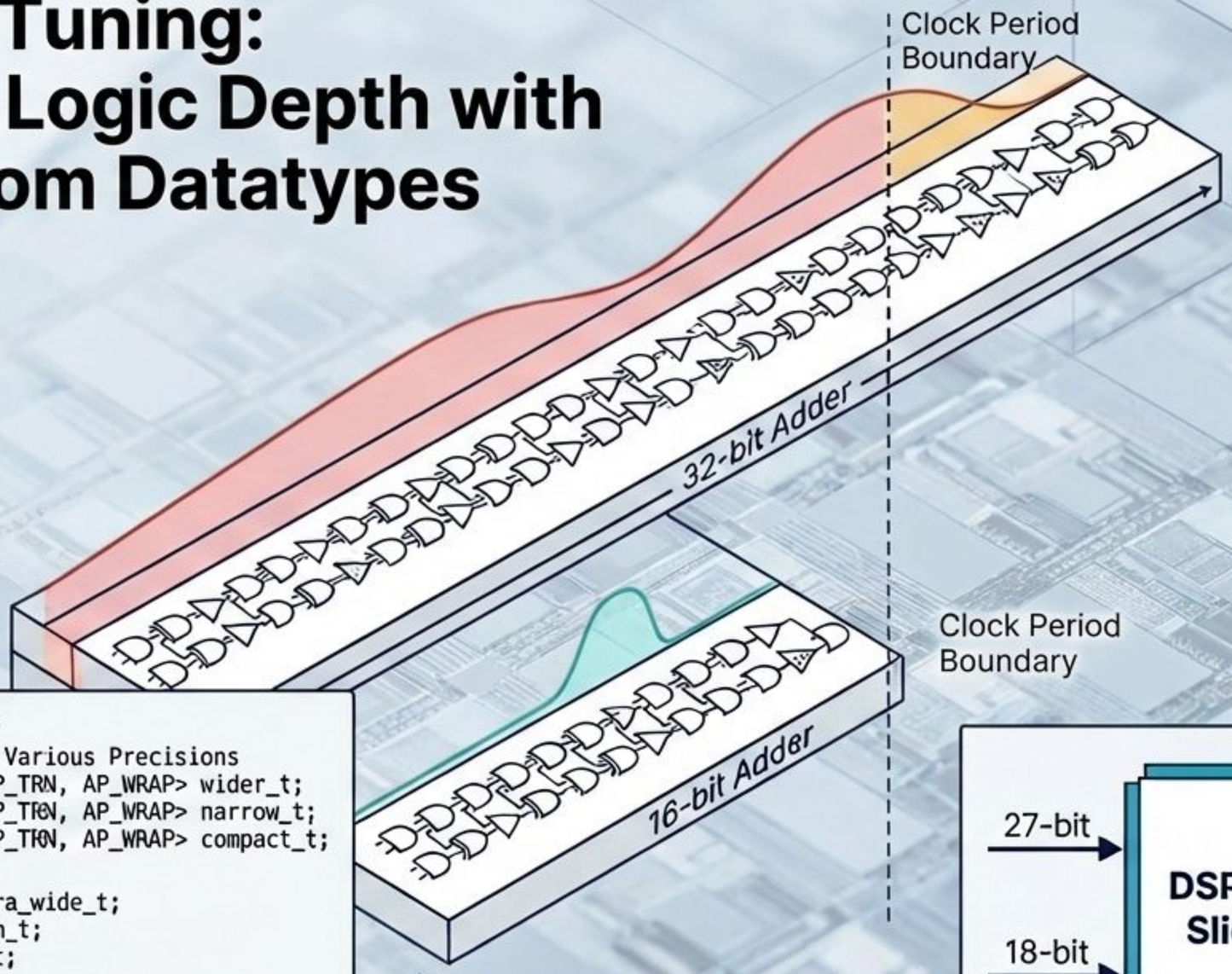


What is Backpressure ? A condition where a downstream component (consumer) cannot process data as fast as it is being produced by an upstream component (producer).

Consequence: Data piles up in intermediate buffers (FIFOs), eventually causing upstream components to stall and wait.

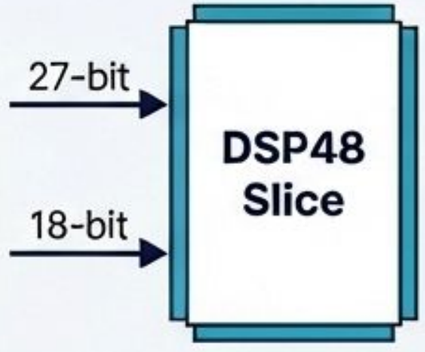
Impact: The entire pipeline's throughput is reduced to the speed of the slowest component, defeating the purpose of high-speed streaming.

Precision Tuning: Shrinking Logic Depth with with Custom Datatypes



All Custom Datatypes Used:

```
// Fixed-point Types with Various Precisions
typedef ap_fixed<22, 8, AP_TRN, AP_WRAP> wider_t;
typedef ap_fixed<16, 4, AP_TRN, AP_WRAP> narrow_t;
typedef ap_fixed<14, 3, AP_TRN, AP_WRAP> compact_t;
// Unsigned Integer Types
typedef ap_uint<1024> ultra_wide_t;
typedef ap_uint<27> medium_t;
typedef ap_uint<18> slim_t;
// Structs for Vectorization
struct vec32_t { narrow_t data[32]; };
struct vec16_t { wider_t data[16]; };
```

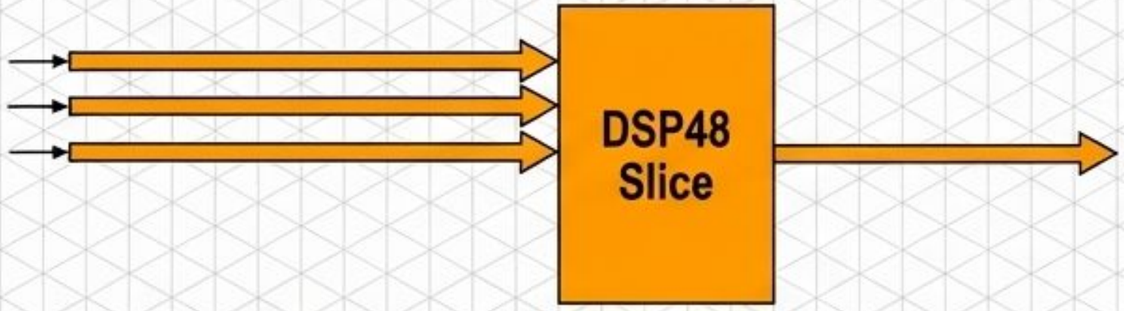


Shrinking variables to 14, 16, or 22 bits ensures multipliers fit exactly into ONE DSP slice.

Cascading multiple slices across the fabric ruins timing.

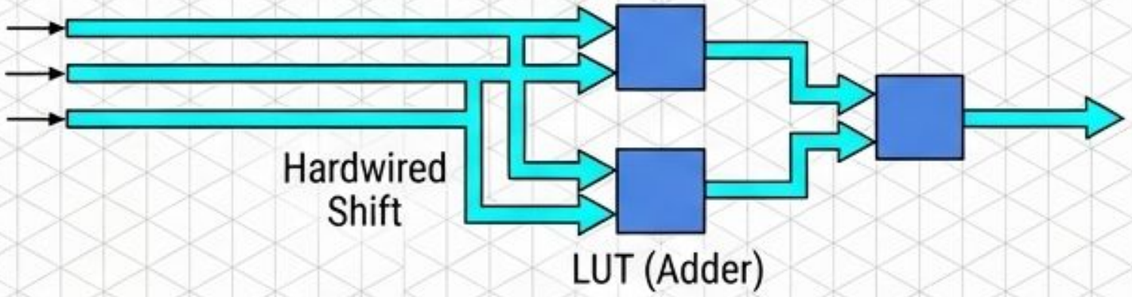
Arithmetic Strength Reduction: Routing vs. Computing

Costly Path: DSP48 Multiplication



Data must route to dedicated DSP columns, causing significant wire delays and consuming valuable resources.

Optimized Path: LUT-based Shift & Add



Bit-shifts are implemented as free, hardwired routing. LUT adders are local, bypassing DSP paths for minimal delay.

Implementation Examples

	Transform	Original Operation		Optimized Operation
1	K1 Transform:	0.5*x0 - 0.25*x1 + 0.125*x2	→	(x0 >> 1) - (x1 >> 2) + (x2 >> 3)
2	K4 Postprocess:	1.25 * x	→	x + (x >> 2)

Bypassing DSP blocks drastically reduces the combinatorial critical path, increasing the maximum clock frequency (F_max).

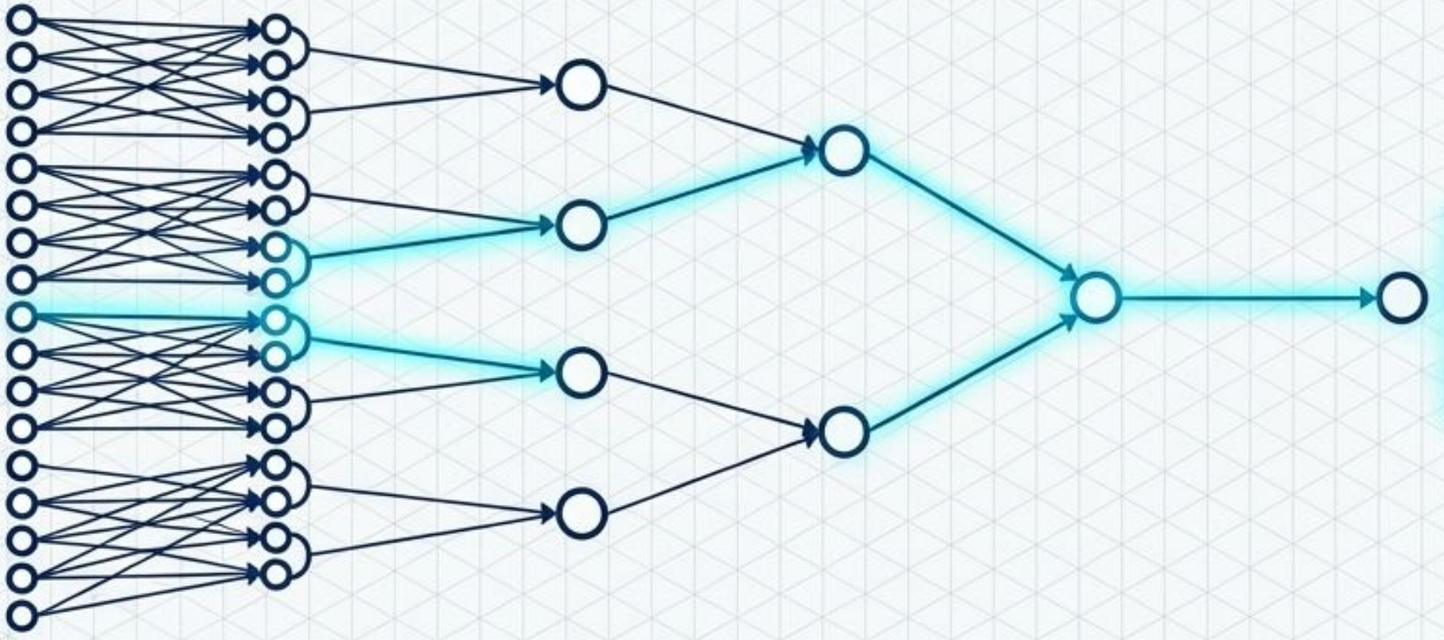
Breaking Loop-Carry Dependencies: The Balanced Binary Adder Tree

```
sum += abs(x)
```

Baseline Loop

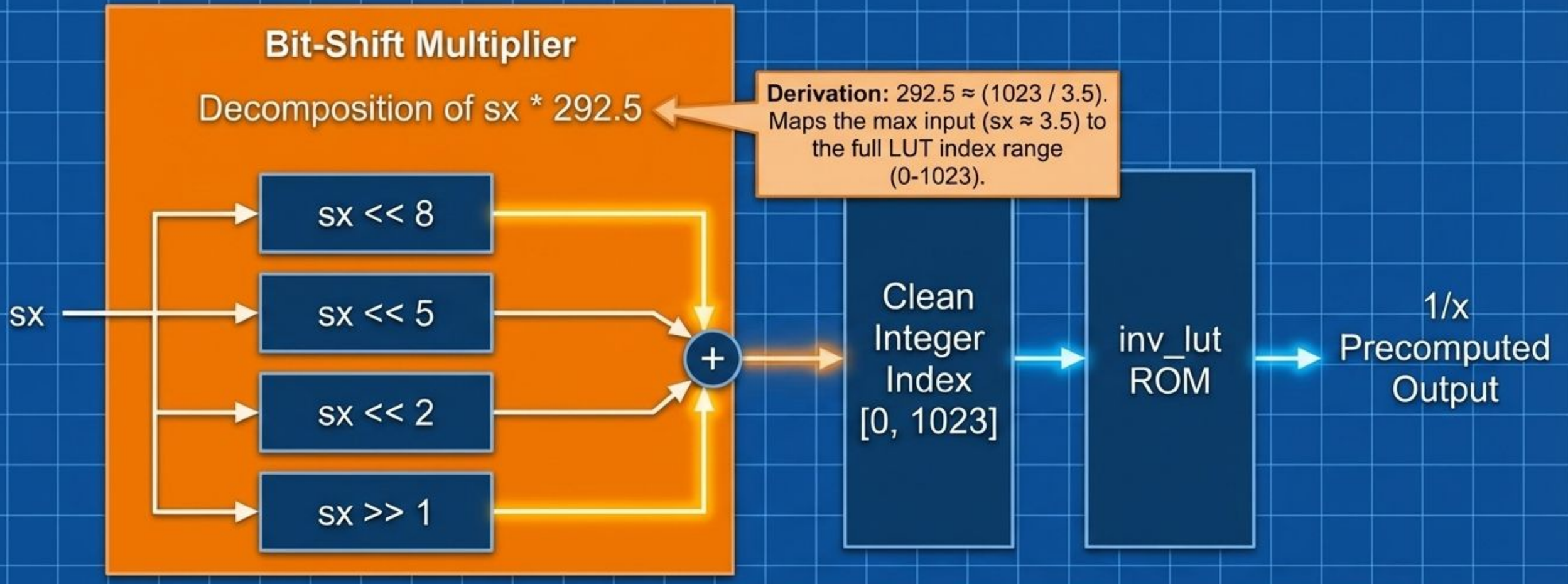
⚠ O(N) Sequential Logic: 32 chained combinational adders. Forces massive clock period.

Optimized Tree



O(log N) Parallel Logic: Electrical signal now only ripples through 5 adders instead of 32. Essential for timing closure when processing 32 elements per cycle.

Eradicating Division: Fractional Indexing via Bit-Shifts



The Hardware Reality: Replacing division with bit-shifts and a ROM lookup significantly reduces latency and hardware area. The precomputed inverse table is indexed by a scaled version of the input, ‘sx’, converting the division problem into a simple memory read.

Localizing Routing: The Hardware Storage Hierarchy

SRL (Shift Register LUT)

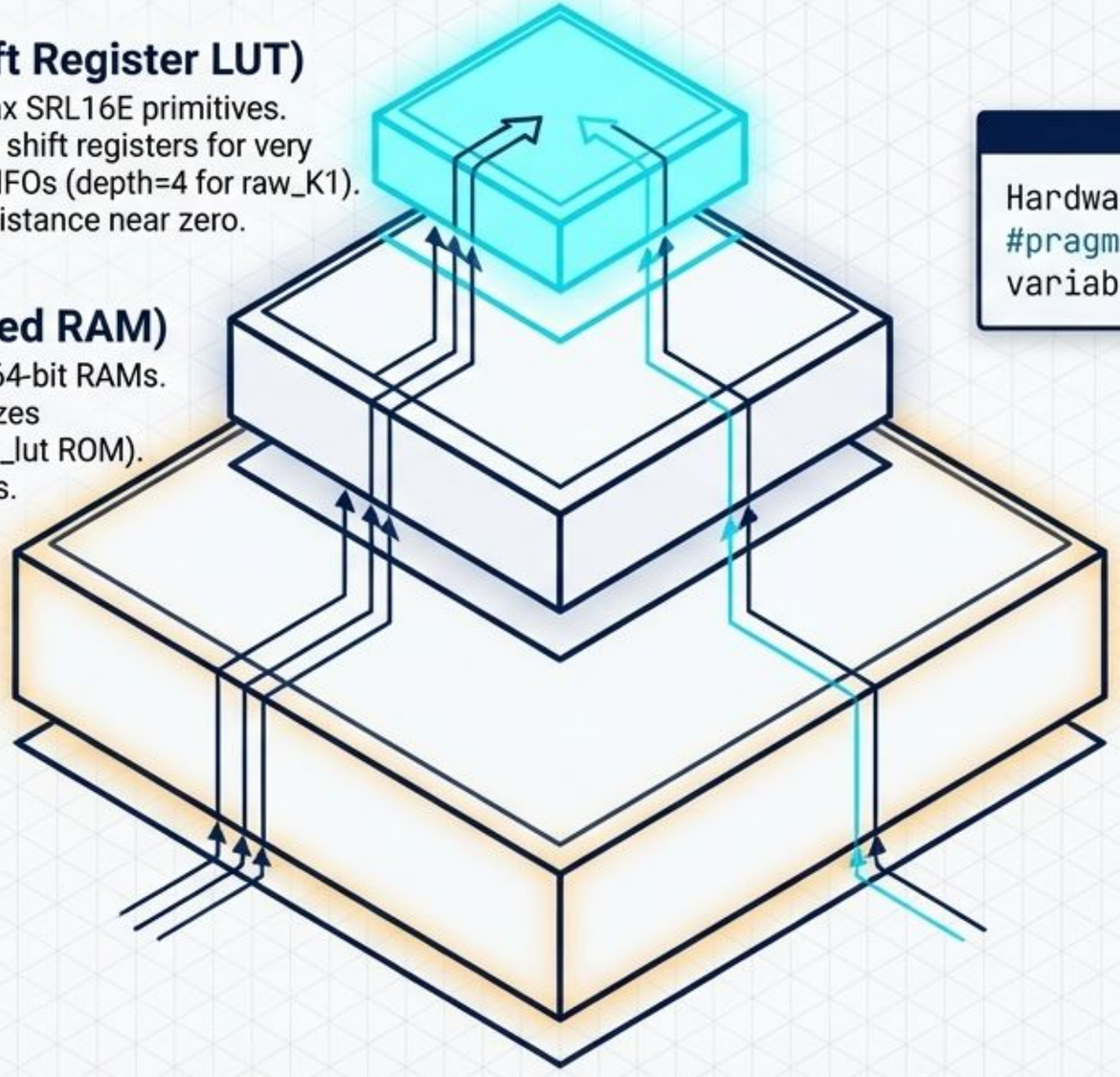
- Uses Xilinx SRL16E primitives.
- Localized shift registers for very shallow FIFOs (depth=4 for raw_K1).
- Routing distance near zero.

LUTRAM (Distributed RAM)

- Uses slice LUTs as tiny 64-bit RAMs.
- Ideal for intermediate sizes (depth=64 for s1_K3, inv_lut ROM).
- Single-cycle local access.

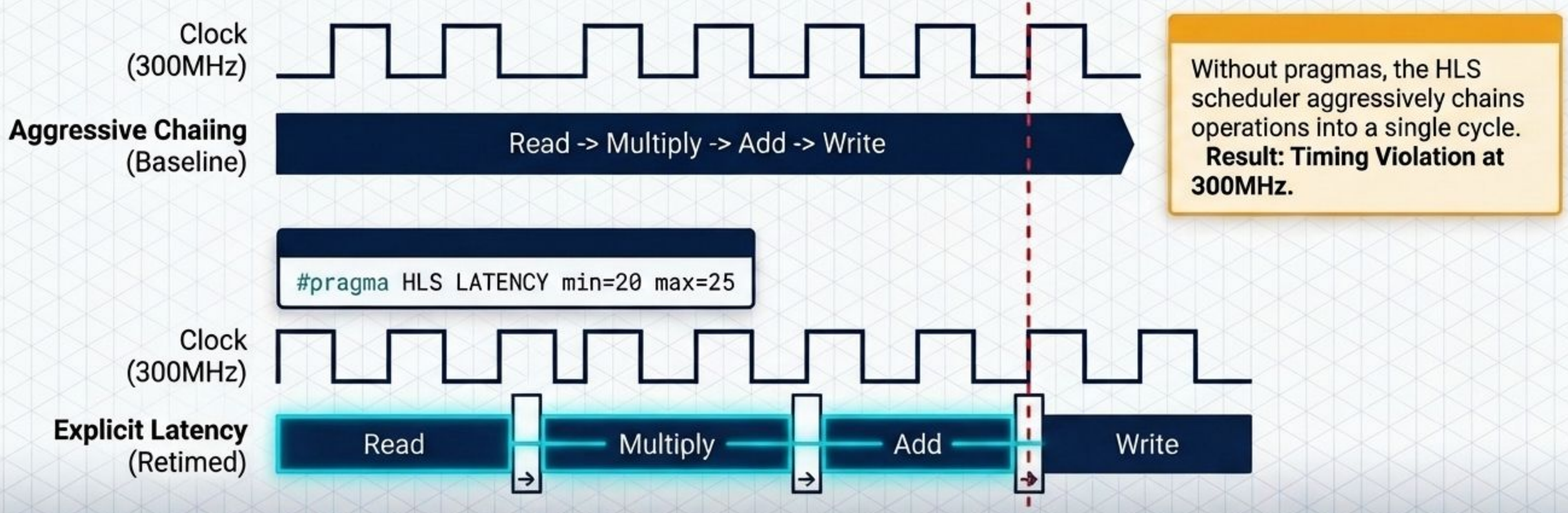
BRAM (Block RAM)

- Large, distant silicon blocks. Routing to these columns creates long wire delays.
- Default for HLS streams, but must be overridden.



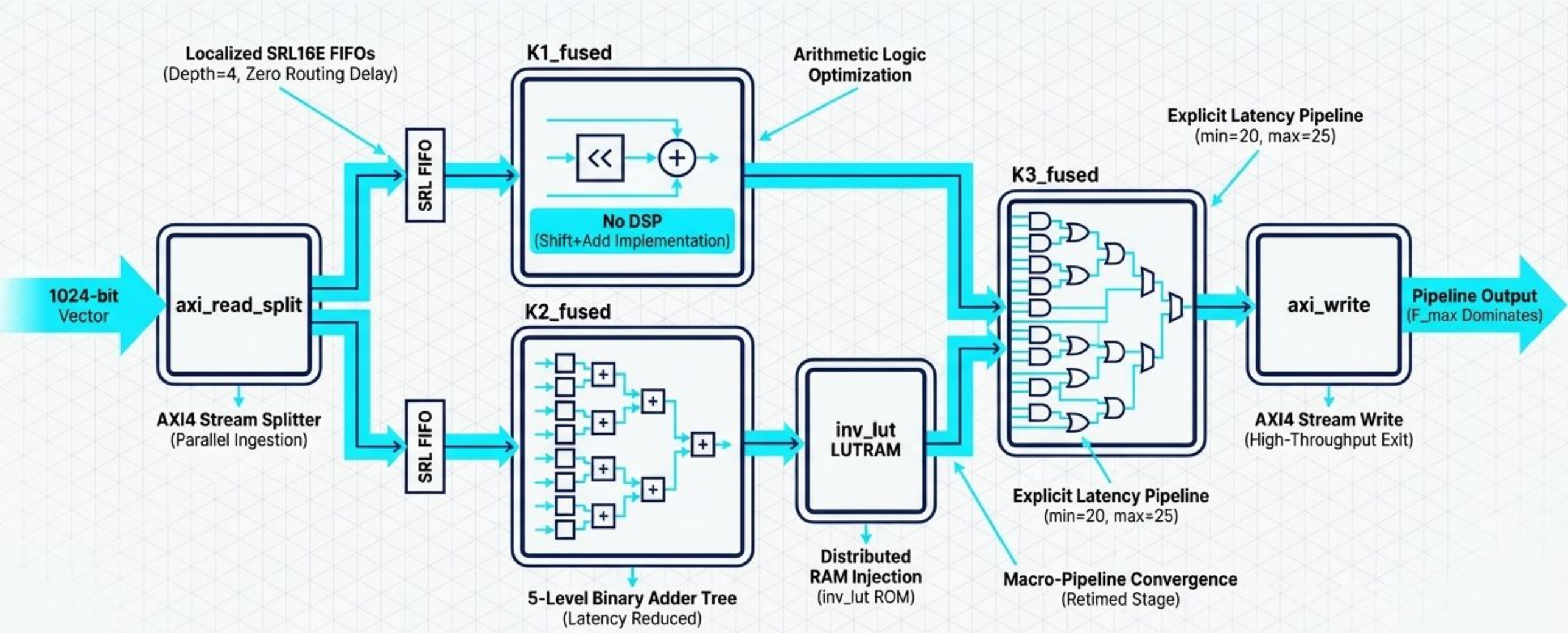
```
Hardware Override:
#pragma HLS BIND_STORAGE
variable=stream type=fifo impl=srl
```

Controlling the Scheduler: Retiming via Explicit Latency



Without pragmas, the HLS scheduler aggressively chains operations into a single cycle. **Result: Timing Violation at 300MHz.**

The Macro-Pipeline Synthesis: From Software to Silicon



The Blueprint Realized: F_max and Massive Throughput

Architectural Achievements

- **Decoupled Memory Access:** Eradicated sequential memory bottlenecks via decoupled `hls::stream` FIFOs.
- **Optimized Datapath:** Bypassed heavy DSPs and dividers using strength reduction and localized LUT indexing.
- **Timing Closure & Pipelining:** Achieved pristine timing closure through explicit retiming and balanced adder trees.

Final Achieved Clock Cycles: 2088
Clock Period: 1.433ns (698.00 MHz)